

Experimental Execution Summary: EMCAD Replication on ClinicDB

Applicant: Audity Ghosh

Selected paper: *EMCAD: Efficient Multi-scale Convolutional Attention Decoding for Medical Image Segmentation*

Model: PVT-EMCAD-B2

Task: Binary polyp segmentation

Dataset: ClinicDB

1. Experimental Objective

I selected EMCAD for experimental verification and used its open-source codebase to reproduce the PVT-EMCAD-B2 segmentation pipeline on the ClinicDB dataset. My objective was to verify that I could successfully configure and execute the released implementation, train the model under an independent computational environment, evaluate the resulting checkpoint, and compare the reproduced performance with the benchmark reported in the paper.

2. Experimental Setup

I conducted the experiment on **Kaggle using a single NVIDIA Tesla T4 GPU**. The released EMCAD codebase was configured for ClinicDB with the PVTv2-B2 encoder and EMCAD decoder.

Component	My experimental setup
Platform	Kaggle
GPU	NVIDIA Tesla T4
Framework	PyTorch 2.10.0+cu128
CUDA available	Yes
timm	1.0.26
Dataset	ClinicDB
Task	Binary polyp segmentation
Model	PVT-EMCAD-B2
Encoder	PVTv2-B2
Encoder initialization	ImageNet-pretrained weights
Input resolution	352×352

Component	My experimental setup
Batch size	4
Optimizer	AdamW
Weight decay	1×10^{-4}
Maximum epochs	200
Gradient clipping	0.5
EMCAD kernel sizes	[1, 3, 5]
Test-set size	62 images

For comparison, the paper reports experiments using **PyTorch 1.11.0 on a single NVIDIA RTX A6000 (48 GB)**. For ClinicDB and related binary segmentation datasets, the authors use 352×352 inputs, AdamW with learning rate and weight decay of 10^{-4} , generally 200 epochs with batch size 16, multi-scale training at {0.75, 1.0, 1.25}, and gradient clipping at 0.5.

Therefore, this experiment should be considered an **implementation-based replication under different computational conditions**, rather than an exact reproduction of the authors' environment. I used a **Kaggle NVIDIA Tesla T4**, which has substantially less GPU memory than the A6000 used in the paper. To make the PVT-EMCAD-B2 training configuration feasible within the available GPU memory, I reduced the batch size from 16 to **4**.

3. Pre-training Execution Checks

Before starting full training, I verified the main components of the pipeline.

The environment successfully detected:

PyTorch: 2.10.0+cu128
 CUDA: True
 GPU: Tesla T4
 timm: 1.0.26

The PVTv2-B2 backbone and EMCAD decoder were also instantiated successfully:

Model pvt_v2_b2 backbone:
 Parameter count: 24,849,856

Model EMCAD decoder:
 Parameter count: 1,913,515

The approximately **1.91M-parameter EMCAD decoder** is consistent with the lightweight decoder configuration described in the paper. The paper reports substantial reductions in decoder parameters and FLOPs relative to CASCADE.

4. Training Execution and Checkpoint Selection

Training was executed for the 200-epoch schedule with checkpointing based on validation Dice.

The later training logs demonstrate stable convergence:

Epoch 198/200
Train Loss : 0.782997
Val Dice : 0.932502
Val IoU : 0.882525
Test Dice : 0.952549
Test IoU : 0.911371
Time : 106.1 s

Epoch 199/200
Train Loss : 0.853302
Val Dice : 0.932743
Val IoU : 0.882786
Test Dice : 0.952946
Test IoU : 0.912031
Time : 106.0 s

Epoch 200/200
Train Loss : 0.789622
Val Dice : 0.932001
Val IoU : 0.881938
Test Dice : 0.952277
Test IoU : 0.910881
Time : 106.4 s

The highest validation Dice occurred earlier than the final epoch:

Best Epoch : 160
Best Val Dice : 0.936713
Test Dice at Best: 0.951631

Accordingly, **epoch 160 rather than epoch 200 was selected as the best checkpoint**. The saved checkpoint contained the model, optimizer and scheduler states together with the best validation score and epoch, allowing the model to be reloaded for a separate evaluation.

This behavior also indicates that extending training to epoch 200 did not produce further validation improvement.

5. Standalone Final Test Verification

After training, I reloaded EMCAD_ClinicDB_best_full.pth and performed a **separate standalone evaluation** over the complete ClinicDB test split.

The evaluation successfully processed all **62/62 test images**:

Checkpoint:

EMCAD_ClinicDB_best_full.pth

Best Epoch: 160

Best Validation Dice: 0.9367125743

Test images: 62

Final ClinicDB Test:

62/62 images evaluated successfully

The independently calculated final metrics were:

Metric	Final result
Dice	0.950151 (95.02%)
IoU	0.907202
Sensitivity	0.954845
Specificity	0.996946
Precision	0.948724
HD95	8.1080
Test images	62

I use **0.950151 as the final replication Dice**, rather than the 0.951631 test value recorded during training, because 0.950151 was obtained by separately loading the selected checkpoint and evaluating it over the complete test set.

6. Verification Against the Published Benchmark

ClinicDB contains 612 images, and the EMCAD experiments use an 80:10:10 train-validation-test split.

For **PVT-EMCAD-B2**, the paper reports a ClinicDB Dice score of **95.21%**, with EMCAD results averaged over five runs.

My independently evaluated result was:

	Published EMCAD	My replication
Model	PVT-EMCAD-B2	PVT-EMCAD-B2
Dataset	ClinicDB	ClinicDB
Input resolution	352×352	352×352
Reported/test Dice	95.21%	95.02%
Absolute difference	—	−0.19 percentage points
Runs	5-run average	1 run
GPU	RTX A6000 48 GB	Tesla T4
Batch size	16	4

The replication result is therefore **very close to the published benchmark**, with an absolute Dice difference of approximately **0.195 percentage points**.

Because the original result is a five-run average and my experiment is a single run conducted under different hardware, software, batch-size, and learning-rate conditions, I do not interpret the small difference as an exact like-for-like benchmark comparison. Rather, the result verifies that the released implementation can reproduce performance in the vicinity of the published ClinicDB result under a substantially different execution environment.

7. Execution Notes and Limitations

Several practical issues were identified during replication.

Hardware constraint. The original experiments used an RTX A6000 with 48 GB memory, whereas my experiment used a Kaggle Tesla T4. I therefore used a smaller batch size of 4.

Software-environment difference. My experiment used PyTorch 2.10.0+cu128 rather than the paper's PyTorch 1.11.0 environment.

Single-run verification. The published EMCAD numbers are averaged over five runs, whereas the current replication consists of one complete training run.

8. Execution Outcome

The experiment successfully verified the major components of the EMCAD codebase on ClinicDB:

Codebase configuration → model initialization → pretrained encoder loading → 200-epoch training → validation-based checkpoint selection → checkpoint reload → standalone 62-image test evaluation → benchmark comparison

The final independently verified **95.02% Dice** closely approached the paper's **95.21% five-run result**, with a difference of approximately **0.19 percentage points**, despite the different GPU, software environment, batch size, learning rate, and number of runs.

This experiment therefore provided successful practical verification of the released PVT-EMCAD-B2 implementation on ClinicDB while also identifying several experimental controls that I would tighten in a publication-oriented replication.